Homework 6

6.3200.)

a.

Define

$$y = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$F = \begin{bmatrix} f_0(x_0) & f_1(x_0) & ... & f_m(x_0) \\ f_0(x_1) & f_1(x_1) & ... & f_m(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ f_0(x_n) & f_1(x_n) & ... & f_m(x_n) \end{bmatrix} \in \mathbb{R}^{(n+1)\times(m+1)}$$

$$\alpha = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_m \end{bmatrix} \in \mathbb{R}^{m+1}$$

Then we have

$$\min_{\alpha} \|y - F\alpha\|_2^2 = \min_{\alpha_0,\alpha_1,...,\alpha_n} \sum_{i=0}^{n} \|y_i - (\alpha_0 f_0(x_i) + \alpha_1 f_1(x_i) + ... + \alpha_m f_m(x_i))\|^2$$

$$= \min_{\alpha_0,\alpha_1,...,\alpha_n} \sum_{i=0}^{n} \|y_i - g(x_i)\|^2$$

Therefore, we've expressed the problem in the form $\min \|y - F\alpha\|_2^2$ for some known vector y, known matrix F, and unknown vector α.

b.

Given that n is a multiple of m and there are $n+1$ data points that are evenly spaced in x from 0 to 12 inclusive, so that gap between points is $\frac{12}{n}$, we want to show that F as defined above is full rank. As $F \in \mathbb{R}^{(n+1)\times(m+1)}$ and $n \geq m$, to prove that F is full rank, we just need to show that the columns of F are linearly independent. We can do this with an induction argument on the columns of F going from right to left. Base case: the singleton set of the rightmost column of F is linearly independent. Inductive step: assume that the k rightmost columns of F are linearly independent. We'll prove that the $k+1$ rightmost columns of F must be linearly independent as well. To prove this, we just have to show that the $k+1$th rightmost column of F exists outside the span of the k rightmost columns. The set of the k rightmost columns of F can be represented as

$$\{ \begin{bmatrix} f_{m+1-k}(x_0) \\ f_{m+1-k}(x_1) \\ \vdots \\ f_{m+1-k}(x_n) \end{bmatrix}, \begin{bmatrix} f_{m+2-k}(x_0) \\ f_{m+2-k}(x_1) \\ \vdots \\ f_{m+2-k}(x_n) \end{bmatrix}, ..., \begin{bmatrix} f_m(x_0) \\ f_m(x_1) \\ \vdots \\ f_m(x_n) \end{bmatrix} \}$$

Now consider x_i. We know that the distance between points is $\frac{12}{n}$ and that $n = zm$ for some $z \in \mathbb{N}$, so for $i \in \{0, 1, ..., n\}$

$$x_i = \frac{12}{n} i = \frac{12}{zm} i$$

This means that since $(m-k)z \in \{0, 1, ..., n\}$, we have

$$x_{(m-k)z} = \frac{12}{zm}(m-k)z = \frac{12}{m}(m-k)$$

Notice that for $j \in \{m+2-k, ..., m\}$, we have

$$x_{(m-k)z} = \frac{12}{m}(m-k) < \frac{12}{m}(j-1) \Rightarrow f_j(x_{(m-k)z}) = 0$$

and for $j = m + 1 - k$,

$$x_{(m-k)z} = \frac{12}{m}(m-k) = \frac{12}{m}(j-1) \Rightarrow f_j(x_{(m-k)z}) = \frac{mx_{(m-k)z}}{12} - (j-1) = (m-k) - (m+1-k-1) = 0$$

However, for $j = m - k$ (representing the $k+1$th rightmost column),

$$x_{(m-k)z} = \frac{12}{m}(m-k) = \frac{12}{m}j \Rightarrow f_j(x_{(m-k)z}) = \frac{mx_{(m-k)z}}{12} - (j-1) = (m-k) - (m-k-1) = 1$$

In other words, there exists a row where the k rightmost columns have a 0 entry, but the $k+1$th rightmost column has a 1 entry. This means the $k+1$th rightmost column is outside the span of the k rightmost columns, so the $k+1$ rightmost columns of F are linearly independent. This completes our induction argument, thus all of the columns of F are linearly independent, so F is full rank.

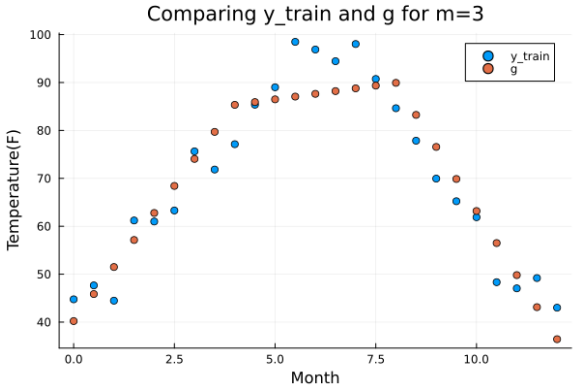
Julia Code.

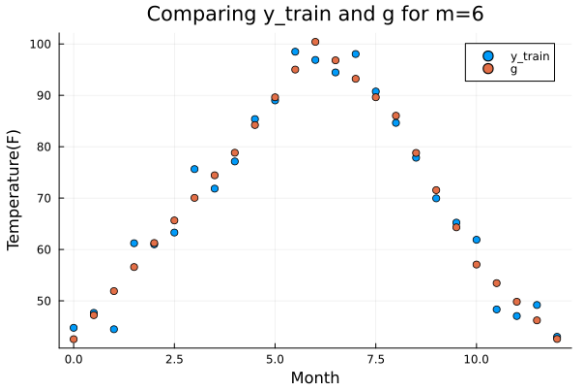
```julia
using LinearAlgebra, Plots

#Question 1.)

include("readclassjson.jl")
data = readclassjson("tempdata.json")

y_train = data["y_train"]
x_train = data["x_train"]
y_test = data["y_test"]
x_test = data["x_test"]

#Part C
function compute_F(x_array, m)
    F = zeros(length(x_array), m + 1)
    F[:, 1] .= 1.0
    for j in 1:m
        for (i, x) in enumerate(x_array)
            if x < ((j-1) * 12.0 / m)
                F[i, j + 1] = 0.0
            elseif x <= (j * 12.0 / m)
                F[i, j + 1] = m*(x/12.0) - (j-1)
            else
                F[i, j + 1] = 1.0
            end
        end
    end
    return F
end

m_values = [3, 6, 12, 24]
errors = zeros(length(m_values))
alpha_dict = Dict{Int, Vector{Float64}}()
for (i, m) in enumerate(m_values)
    F = compute_F(x_train, m)
    alpha = F \ y_train
    g = F*alpha
    plot(x_train, y_train, label = "y_train", seriestype=:scatter)
    plot!(x_train, g, label = "g", seriestype=:scatter)
    xlabel!("Month")
    ylabel!("Temperature(F)")
    title!("Comparing y_train and g for m=$(m)")
    savefig("hw6_q1c_m=$(m).png")

    errors[i] = norm(y_train-g)^2
    alpha_dict[m] = alpha
end

```

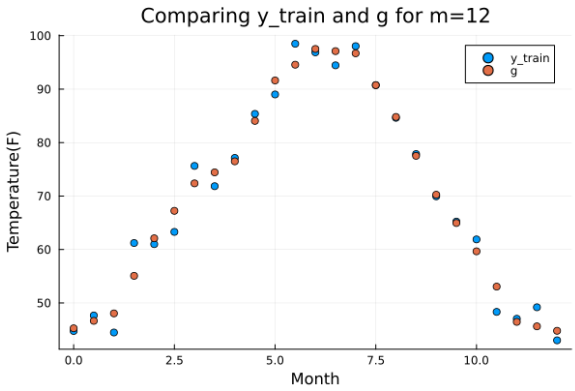
```
#Part D
plot(m_values, errors, label = "Continuous Extension of Error")
plot!(m_values, errors, label = "Error Points", seriestype=:scatter)
xlabel!("m")
ylabel!("Error")
title!("Minimal Squared 2-Norm Error as a function of m")
savefig("hw6_q1d.png")


#Part E
test_errors = zeros(length(m_values))
for (i, m) in enumerate(m_values)
    alpha = alpha_dict[m]
    F = compute_F(x_test, m)
    g = F * alpha
    test_errors[i] = norm(y_test-g)^2
end
plot(m_values, test_errors, label = "Continuous Extension of Test Error")
plot!(m_values, test_errors, label = "Test Error Points", seriestype=:scatter)
xlabel!("m")
ylabel!("Error")
title!("Test Error as a function of m")
savefig("hw6_q1e.png")
```

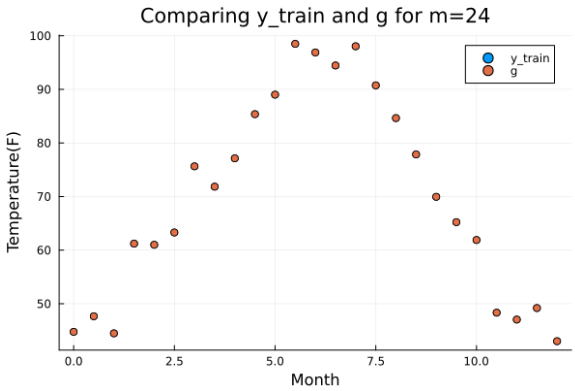
c.

This question was done in Julia, the code is above above and the plots are below. Note that for each given m, F is full rank and skinny, so we could solve $\min_\alpha \|y - F\alpha\|_2^2$ using least squares.



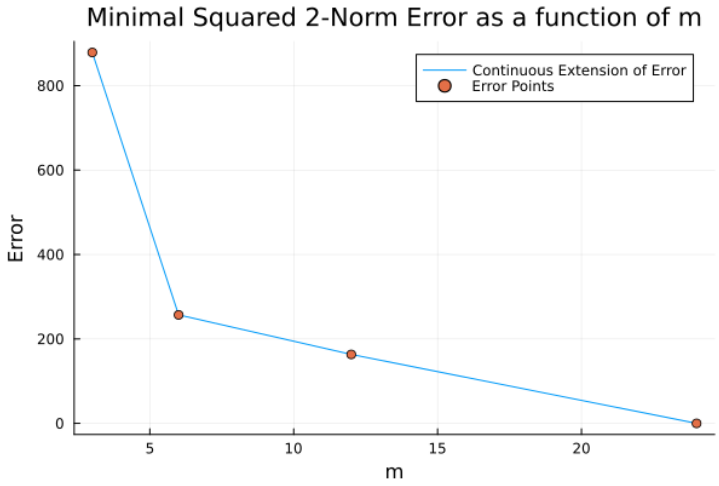






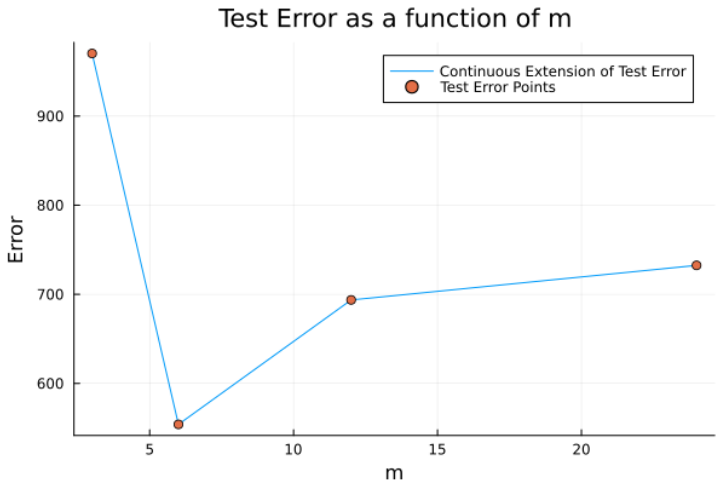
d.

This question was done in Julia, the code is above above and the plot is below. As can be seen in the plot, adding complexity from $m = 3$ to $m = 6$ decreased the error significantly more than adding complexity beyond $m = 6$. Therefore, there is a point at $m = 6$ where adding more complexity to the model offers clearly diminishing returns.



e.

This question was done in Julia, the code is above above and the plot is below. As can be seen in the plot, the test error is decreasing for $m < 6$ and increasing for $m > 6$. This means that adding complexity to the model when $m < 6$ helps make it more accurate (decreasing test error), but adding complexity to the model when $m > 6$ causes the model to overfit to the training data (increasing test error). This explains why there was diminishing returns in part d, as the diminishing returns come from overfitting to the training data as opposed to making a more accurate model.



7.1040.)

a.

First, note that minimizing E is equivalent to minimizing the following function

$$\sum_{i=1}^{N}(f(t_i)-y_i)^2 = \|f(t)-y\|_2^2$$

To translate this problem into Gauss-Newton, we define

$$\|r(p)\|^2 = \|f(t)-y\|_2^2$$

$$r_i(p)^2 = (f(t_i)-y_i)^2$$

The Gauss-Newton algorithm will find p that minimizes $\|r(p)\|^2$ (which also minimizes E). We start the algorithm by finding a good first guess for $p^{(1)}$. We can do this by looking at the plot of y and guessing appropriate $a^{(1)}, \mu^{(1)}, \sigma^{(1)}$ for the amplitude, center and width of y respectively. Since we have a first guess, we can now start iterating through the algorithm. An iteration finds a new guess $p^{(k+1)}$ given the current guess $p^{(k)}$. We linearize near current $p^{(k)}$:

$$r(p) \approx r(p^{(k)}) + Dr(p^{(k)})(p-p^{(k)})$$

We then write the linearized approximation as

$$r(p^{(k)}) + Dr(p^{(k)})(p-p^{(k)}) = A^{(k)}p - b^{(k)}$$

where

$$A^{(k)} = Dr(p^{(k)}) = \begin{bmatrix} \frac{\partial r_1}{\partial a} & \frac{\partial r_1}{\partial \mu} & \frac{\partial r_1}{\partial \sigma} \\ \frac{\partial r_2}{\partial a} & \frac{\partial r_2}{\partial \mu} & \frac{\partial r_2}{\partial \sigma} \\ \vdots & \vdots & \vdots \\ \frac{\partial r_N}{\partial a} & \frac{\partial r_N}{\partial \mu} & \frac{\partial r_N}{\partial \sigma} \end{bmatrix} = \begin{bmatrix} \exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) & \frac{2a^{(k)}(t_1-u^{(k)})}{\sigma^{(k)2}}\exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) & \frac{2a^{(k)}(t_1-u^{(k)})^2}{\sigma^{(k)3}}\exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) \\ \exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) & \frac{2a^{(k)}(t_2-u^{(k)})}{\sigma^{(k)2}}\exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) & \frac{2a^{(k)}(t_2-u^{(k)})^2}{\sigma^{(k)3}}\exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) \\ \vdots & \vdots & \vdots \\ \exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) & \frac{2a^{(k)}(t_N-u^{(k)})}{\sigma^{(k)2}}\exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) & \frac{2a^{(k)}(t_N-u^{(k)})^2}{\sigma^{(k)3}}\exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) \end{bmatrix}$$

$$b^{(k)} = Dr(p^{(k)})p^{(k)} - r(p^{(k)}) = A^{(k)}\begin{bmatrix} a^{(k)} \\ \mu^{(k)} \\ \sigma^{(k)} \end{bmatrix} - \begin{bmatrix} a^{(k)}\exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) - y_1 \\ a^{(k)}\exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) - y_2 \\ \vdots \\ a^{(k)}\exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) - y_N \end{bmatrix}$$

$$= \begin{bmatrix} a^{(k)}\exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) + \mu^{(k)}\frac{2a^{(k)}(t_1-u^{(k)})}{\sigma^{(k)2}}\exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) + \sigma^{(k)}\frac{2a^{(k)}(t_1-u^{(k)})^2}{\sigma^{(k)3}}\exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) \\ a^{(k)}\exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) + \mu^{(k)}\frac{2a^{(k)}(t_2-u^{(k)})}{\sigma^{(k)2}}\exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) + \sigma^{(k)}\frac{2a^{(k)}(t_2-u^{(k)})^2}{\sigma^{(k)3}}\exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) \\ \vdots \\ a^{(k)}\exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) + \mu^{(k)}\frac{2a^{(k)}(t_N-u^{(k)})}{\sigma^{(k)2}}\exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) + \sigma^{(k)}\frac{2a^{(k)}(t_N-u^{(k)})^2}{\sigma^{(k)3}}\exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) \end{bmatrix} - \begin{bmatrix} a^{(k)}\exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) - y_1 \\ a^{(k)}\exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) - y_2 \\ \vdots \\ a^{(k)}\exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) - y_N \end{bmatrix}$$

$$= \begin{bmatrix} \frac{2a^{(k)}t_1(t_1-u^{(k)})}{\sigma^{(k)2}} \exp(-\frac{(t_1-\mu^{(k)})^2}{\sigma^{(k)2}}) + y_1 \\ \frac{2a^{(k)}t_2(t_2-u^{(k)})}{\sigma^{(k)2}} \exp(-\frac{(t_2-\mu^{(k)})^2}{\sigma^{(k)2}}) + y_2 \\ \vdots \\ \frac{2a^{(k)}t_N(t_N-u^{(k)})}{\sigma^{(k)2}} \exp(-\frac{(t_N-\mu^{(k)})^2}{\sigma^{(k)2}}) + y_N \end{bmatrix}$$

We then approximate the non-linear least squares problem with a linear least squares problem:

$$\|r(p)\|^2 \approx \|A^{(k)}p - b^{(k)}\|^2$$

Which gives us our next guess $p^{(k+1)}$

$$p^{(k+1)} = (A^{(k)\intercal}A^{(k)})^{-1}A^{(k)\intercal}b^{(k)}$$

We continue iterating until convergence. Convergence isn't guaranteed to happen, but should happen with a good first guess.

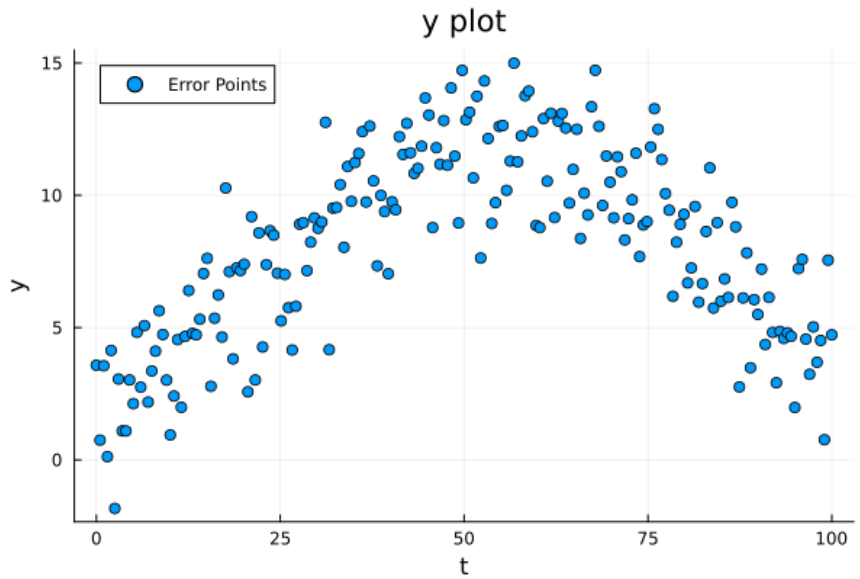
b.

This question was done in Julia, the code, plots and explanation are all below.

```julia
using LinearAlgebra, Plots
using Statistics

#Question 2.)
#Part B

include("readclassjson.jl")
data = readclassjson("gauss_fit_data.json")

t = data["t"]
y = data["y"]
N = data["N"]

plot(t, y, label = "Error Points", seriestype=:scatter)
xlabel!("t")
ylabel!("y")
title!("y plot")
savefig("hw6_q2b_i.png")

function linearize(p)
    a, mu, sigma = p
    A = zeros(N, 3)
    b = zeros(N)
    r = zeros(N)
    for i in 1:N
        ti = t[i]
        fi = f(ti, a, mu, sigma)
        r[i] = fi - y[i]
        A[i, 1] = exp(-(ti - mu)^2 / sigma^2)
        A[i, 2] = fi * (2 * (ti - mu) / sigma^2)
        A[i, 3] = fi * (2 * (ti - mu)^2 / sigma^3)
        b[i] = fi * (2 * ti * (ti - mu) / sigma^2) + y[i]
    end
    return A, b, r
end

```

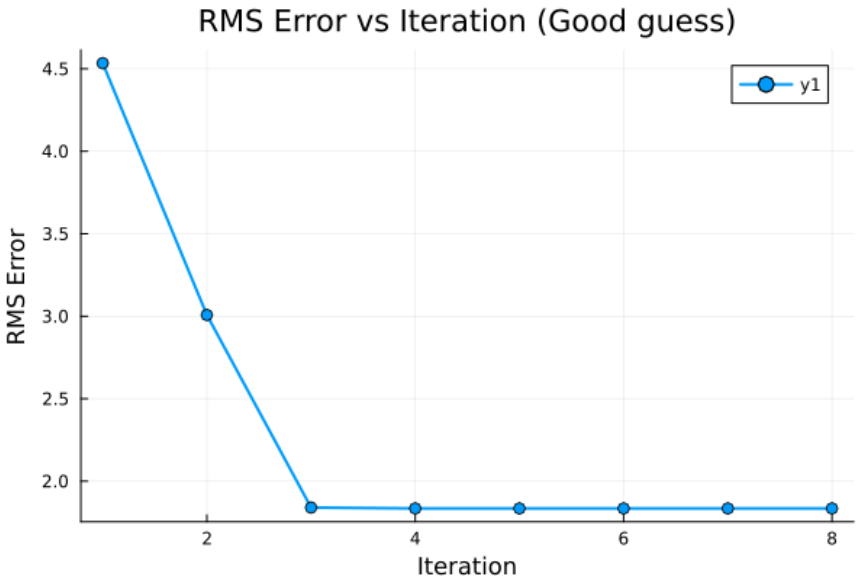
```julia
function gauss_newton(p_start)
    maxIterations = 50
    tolerance = 1e-8
    p = copy(p_start)
    rms_errors = Float64[]
    for k in 1:maxIterations
        A, b, r = linearize(p)
        p_new = A \ b
        push!(rms_errors, sqrt(mean(r.^2)))
        if norm(p_new - p) < tolerance
            println("Converged after $k iterations")
            return p_new, rms_errors
        end
        p = p_new
    end
    println("Didn't converge")
    return p, rms_errors
end

p_good = [15, 50, 20]
p_diff = [13, 55, 25]
p_bad = [8, 0, 100]
print(p_good, p_diff, p_bad)
guesses = Dict(
    "Good guess" => p_good,
    "Different guess" => p_diff,
    "Bad guess" => p_bad
)

f(t, a, mu, sigma) = a * exp(-(t - mu)^2 / sigma^2)

for (label, p_start) in guesses
    p_final, rms_errors = gauss_newton(p_start)

    plot(1:length(rms_errors), rms_errors, xlabel="Iteration", ylabel="RMS Error", title="RMS Error vs Iteration ($label)", lw=2, marker=:o)
    savefig("hw6_q2b_ii_$(label[1:3]).png")

    fitted_gaussian = [f(tt, p_final...) for tt in t]
    scatter(t, y, label="Data", xlabel="t", ylabel="y", title="Gaussian Fit ($label)", legend=:top)
    plot!(t, fitted_gaussian, lw=2, label="Fitted Gaussian")
    savefig("hw6_q2b_iii_$(label[1:3]).png")
end
```

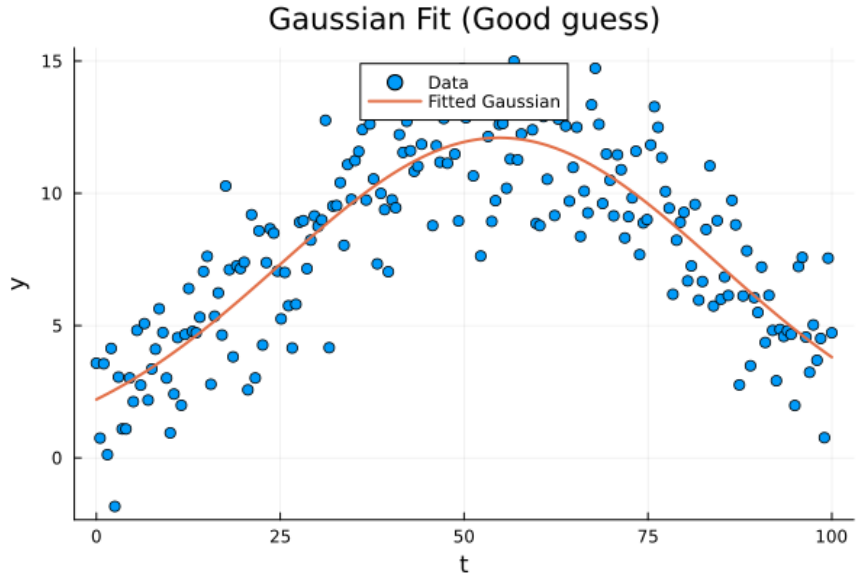
First, we plotted y to visually estimate initial parameter guesses.



We can see that t approximately ranges from 0 to 100. If we assume μ is in the middle, we get an initial guess of $\mu = 50$. Further, if we assume the plot of y contains 4 to 6 standard deviations, we get an initial guess of $\sigma = \frac{100}{5} = 20$. Finally, we see that the max of y is approximately 15, which we can use for an initial guess of a. Thus, our first initial guess is

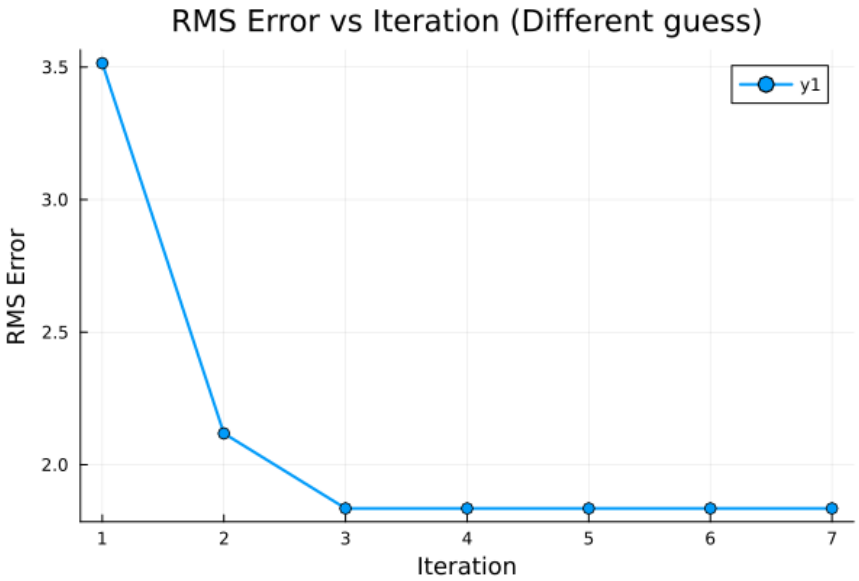
$$\begin{bmatrix} a^{(0)} & \mu^{(0)} & \sigma^{(0)} \end{bmatrix} = \begin{bmatrix} 15 & 50 & 20 \end{bmatrix}$$

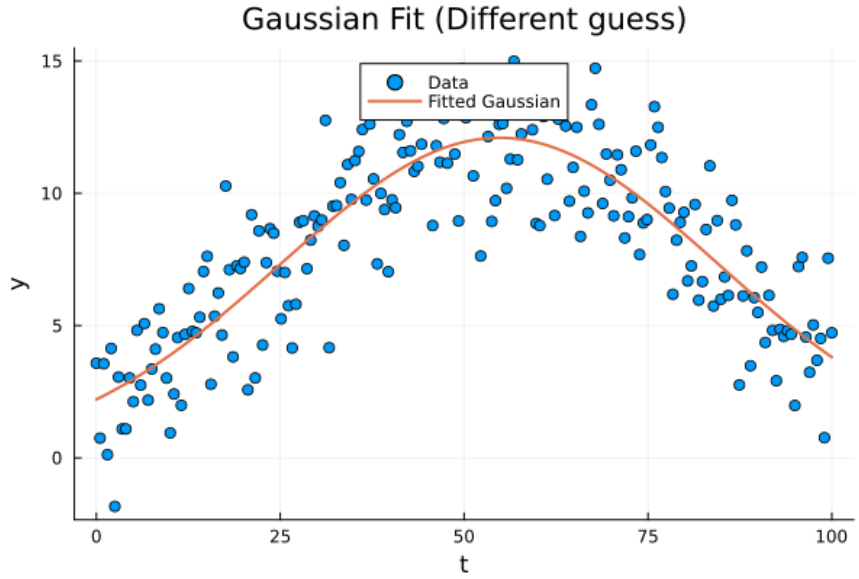




We can use similar logic to generate a different reasonable initial guess. We can argue that the max of y is around $t = 55$, which could be an initial guess for μ. If we now assume the plot of y contains strictly 4 standard deviations, we get an initial guess of $\sigma = \frac{100}{4} = 25$. Finally, we can choose an initial guess for a to be a bit below the max of y, as that max might overshoot, so an initial guess of $a = 13$. Thus, our different reasonable initial guess is

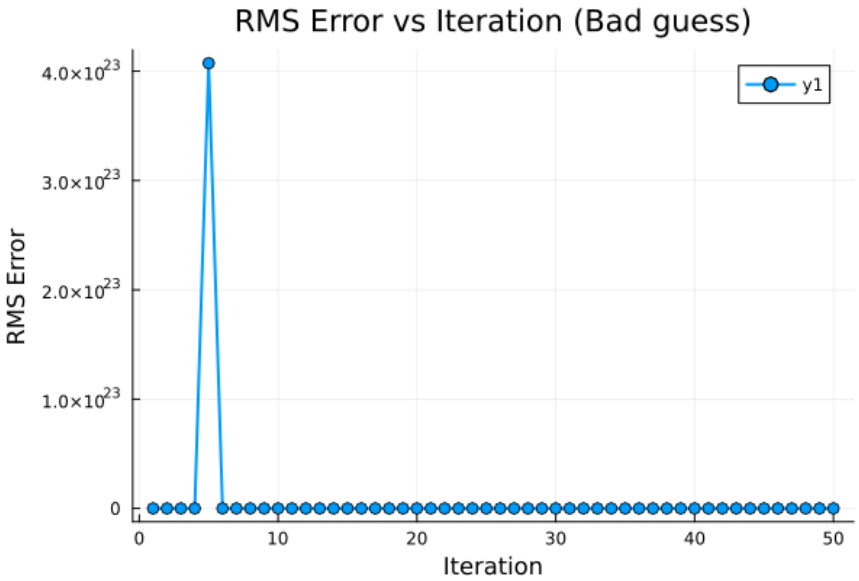
$$\begin{bmatrix} a^{(0)} & \mu^{(0)} & \sigma^{(0)} \end{bmatrix} = \begin{bmatrix} 13 & 55 & 25 \end{bmatrix}$$

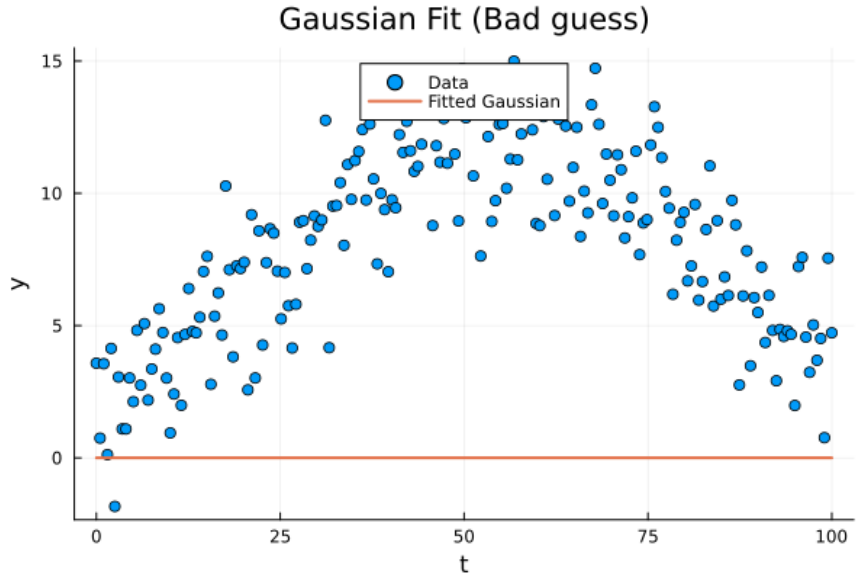




We now want an unreasonable initial guess. We can have an initial guess of $\mu = 0$, as that's the leftmost value of t, so it should be very far from the center. It's unreasonable to assume the plot of y contains strictly 1 standard deviation, which would be an initial guess of $\sigma = 100$. Finally, we halve our reasonable initial guess for a, so an initial guess of $a = 8$. *Note this set of initial guesses didn't converge, which is why the following plots look significantly different from the previous ones.* Thus, our unreasonable initial guess is

$$\begin{bmatrix} a^{(0)} & \mu^{(0)} & \sigma^{(0)} \end{bmatrix} = \begin{bmatrix} 8 & 0 & 100 \end{bmatrix}$$





As can be seen in the three different sets of results, the reasonable initial guesses both converged to the same gaussian approximation, while the unreasonable initial guess didn't converge at all. Further, both reasonable initial guesses converged within 3 iterations of the Gauss-Newton algorithm, while the unreasonable initial guess still hadn't converged after 50 iterations of the Gauss-Newton algorithm. This implies that to converge to an appropriate solution, there's a fair amount of leniency when choosing initial guesses, but the initial guess must be at least somewhat reasonable.

8.1460.)

a.

Define

$$G = \begin{bmatrix} -1 & 1 & 0 & 0 & \dots & 0 & 0 \\ 0 & -1 & 1 & 0 & \dots & 0 & 0 \\ 0 & 0 & -1 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & -1 & 1 \end{bmatrix} \in \mathbb{R}^{(n-1)\times n}$$

Then

$$Gz = \begin{bmatrix} z_2 - z_1 \\ z_3 - z_2 \\ z_4 - z_3 \\ \vdots \\ z_n - z_{n-1} \end{bmatrix} = z^{\text{der}}$$

b.

We can rewrite the set K as $K = \{k_1, k_2, ..., k_{|K|}\}$ where $1 \leq k_1 < k_2 < ... < k_{|K|} \leq 100 = n$. Similarly, rewrite the set K^c as $K^c = \{k_1^c, k_2^c, ..., k_{n-|K|}^c\}$ where $1 \leq k_1^c < k_2^c < ... < k_{n-|K|}^c \leq 100 = n$. Define

$$\tilde{z} = \begin{bmatrix} z_{k_1} \\ z_{k_2} \\ \vdots \\ z_{k_{|K|}} \\ z_{k_1^c} \\ z_{k_2^c} \\ \vdots \\ z_{k_{n-|K|}^c} \end{bmatrix} = \begin{bmatrix} z_K \\ z_{K^c} \end{bmatrix}$$

To clarify, $\tilde{G}$ is a permutation of z where the known entries of z (the ones that must match y) are at the top in z_K and the unknown entries are in z_{K^c}. Let $\tilde{G} = \begin{bmatrix} G_K & G_{K^c} \end{bmatrix}$ be a permutation of the columns of G such that

$$Gz = \tilde{G}\tilde{z} = G_K z_K + G_{K^c} z_{K^c}$$

In other words, the columns of G get permuted with the same ordering as the entries of z so that the matrix vector product remains the same. We want to find z that minimizes $\|Gz\|$ subject to $z_K = y$. As

$$\|Gz\| = \|G_K z_K + G_{K^c} z_{K^c}\|$$

This problem is then equivalent to finding z_{K^c} that minimizes $\|G_{K^c} z_{K^c} - (-G_K y)\|$. Assuming that G_{K^c} is skinny and full rank, this can be solved using least squares. We can then find the optimal z_{K^c}

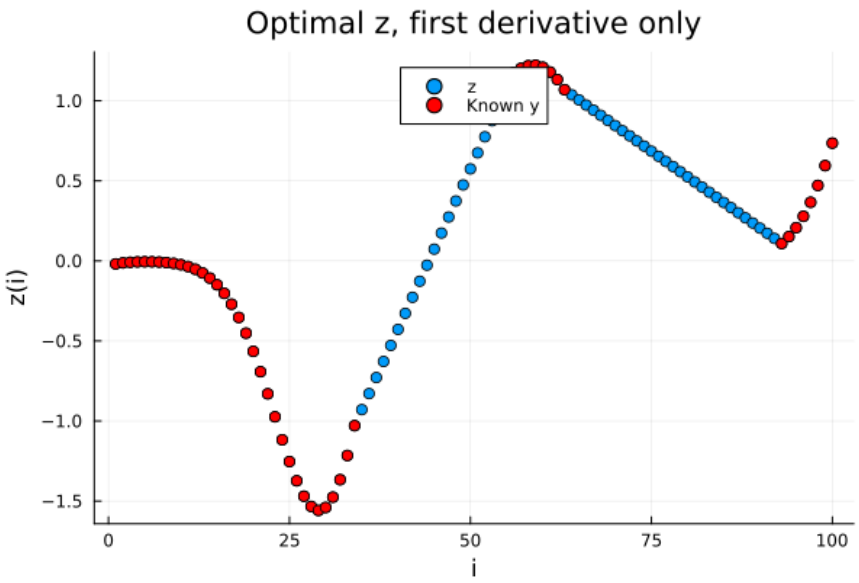
$$\text{optimal } z_{K^c} = (G_{K^c}^\intercal G_{K^c})^{-1} G_{K^c}^\intercal (-G_K y) = -(G_{K^c}^\intercal G_{K^c})^{-1} G_{K^c}^\intercal G_K y$$

We know that $z_K = y$ and we now have the optimal z_{K^c} to minimize $\|Gz\|$, so we can permute the entries back and return the optimal z.

c.

This question was done in Julia, the code and plot can be seen below.

```julia
using LinearAlgebra, Plots

#Question 3.)

include("readclassjson.jl")
data = readclassjson("missing.json")
K = data["known"]
y = data["y"]
n = 100

#Part C

K_c = setdiff(1:n, K)

G = zeros(n - 1, n)
for i in 1:n-1
    G[i, i] = -1
    G[i, i + 1] = 1
end

G_K = G[:, K]
G_K_c = G[:, K_c]

z_K_c = - inv(G_K_c' * G_K_c) * G_K_c' * G_K * y
z_opt = zeros(n)
z_opt[K] = y
z_opt[K_c] = z_K_c

scatter(1:n, z_opt, label="z", xlabel="i", ylabel="z(i)", title="Optimal z, first derivative only", legend=:top)
scatter!(K, y, label="Known y", color=:red)
savefig("hw6_q3c.png")
```



d.

Define

$$H = \begin{bmatrix} 1 & -2 & 1 & 0 & 0 & \dots & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & \dots & 1 & -2 & 1 \end{bmatrix} \in \mathbb{R}^{(n-2)\times n}$$

Then

$$Hz = \begin{bmatrix} z_3 - 2z_2 + z_1 \\ z_4 - 2z_3 + z_2 \\ z_5 - 2z_4 + z_3 \\ \vdots \\ z_n - 2z_{n-1} + z_{n-2} \end{bmatrix} = z^{\text{hes}}$$

e.

To find the optimal z, we can reuse our approach from part b with some multi-objective LS mixed in.

$$\min J_1 + \mu J_2 = \min_z \|Gz\|^2 + \mu\|Hz\|^2 = \min_z \|Gz\|^2 + \|\sqrt{\mu}Hz\|^2 = \min_z \| \begin{bmatrix} G \\ \sqrt{\mu}H \end{bmatrix} z\|^2$$

Let $G' = \begin{bmatrix} G \\ \sqrt{\mu}H \end{bmatrix}$. Then we can define

$$\tilde{G'} = \begin{bmatrix} G'_K & G'_{K^c} \end{bmatrix} = \begin{bmatrix} G_K & G_{K^c} \\ \sqrt{\mu}H_K & \sqrt{\mu}H_{K^c} \end{bmatrix}$$

Then we want to find z that minimizes $\|G'z\|$ subject to $z_K = y$. This lets us reuse our approach from part b. The optimal z_{K^c} is

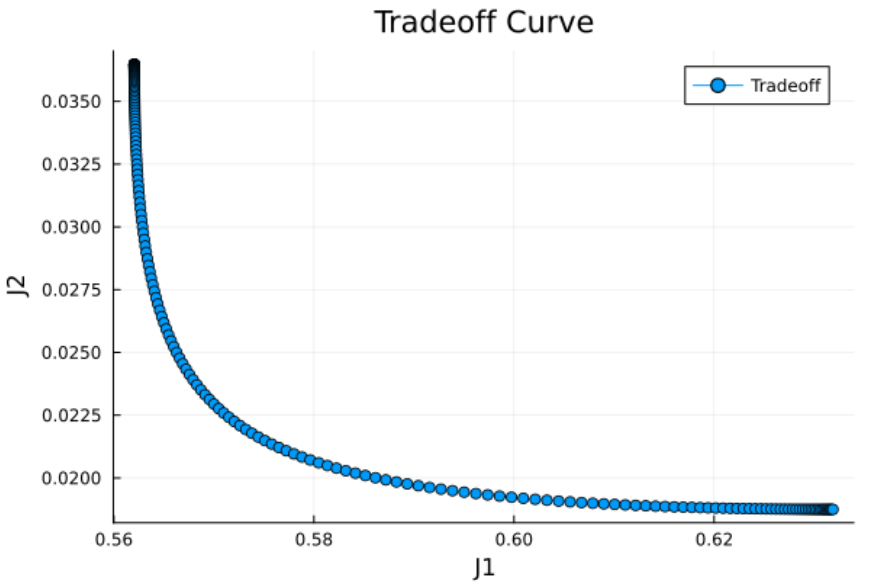
$$\text{optimal } z_{K^c} = -(G'^{\intercal}_{K^c}G'_{K^c})^{-1}G'^{\intercal}_{K^c}G'_K y = -(G^{\intercal}_{K^c}G_{K^c} + \mu H^{\intercal}_{K^c}H_{K^c})^{-1}(G^{\intercal}_{K^c}G_K + \mu H^{\intercal}_{K^c}H_K)y$$

We know that $z_K = y$ and we now have the optimal z_{K^c} to minimize $\|G'z\|$, so we can permute the entries back and return the optimal z.

f.

This question was done in Julia, the code and plot can be seen below. There's two endpoints along the curve, one near the horizontal axis and the other near the vertical axis. The endpoint near the horizontal axis has a high J_1 and low J_2 value, meaning that this endpoint prioritizes having an approximation with minimal second derivative, so the approximation of z looks very smooth and slowly varying.. On the other hand, the endpoint near the vertical axis has a low J_1 and high J_2 value, meaning that this endpoint prioritizes having an approximation with minimal first derivative, so the approxiation of z looks flat and nearly constant.

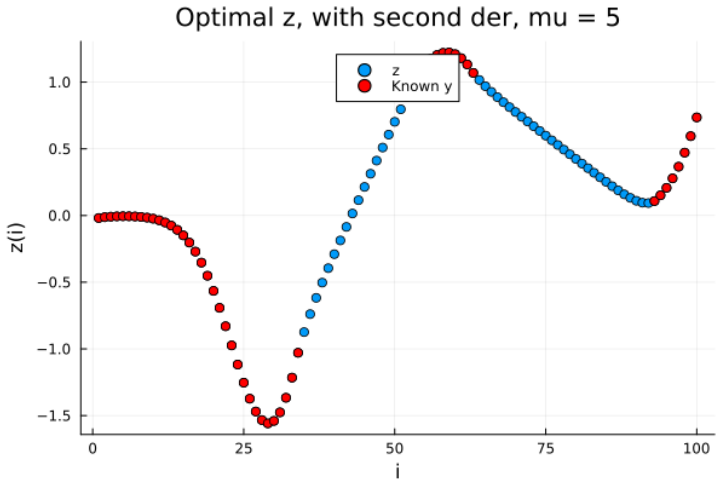
```julia
#Part D

H = zeros(n - 2, n)
for i in 1:n-2
    H[i, i] = 1
    H[i, i + 1] = -2
    H[i, i + 2] = 1
end

H_K = H[:, K]
H_K_c = H[:, K_c]

mu_values = 10 .^ range(-3, 3, length=200)
J1 = []
J2 = []

for mu in mu_values
    z_tradeoff_K_c = - inv(G_K_c' * G_K_c + mu * H_K_c' * H_K_c) * (G_K_c' * G_K + mu * H_K_c' * H_K)* y
    z_tradeoff_opt = zeros(n)
    z_tradeoff_opt[K] = y
    z_tradeoff_opt[K_c] = z_tradeoff_K_c
    push!(J1, norm(G*z_tradeoff_opt)^2)
    push!(J2, norm(H*z_tradeoff_opt)^2)
end

p_tradeoff = plot(J1, J2, marker=:circle,
                  xlabel="J1",
                  ylabel="J2",
                  title="Tradeoff Curve",
                  label="Tradeoff")
savefig("hw6_q3d.png")
```

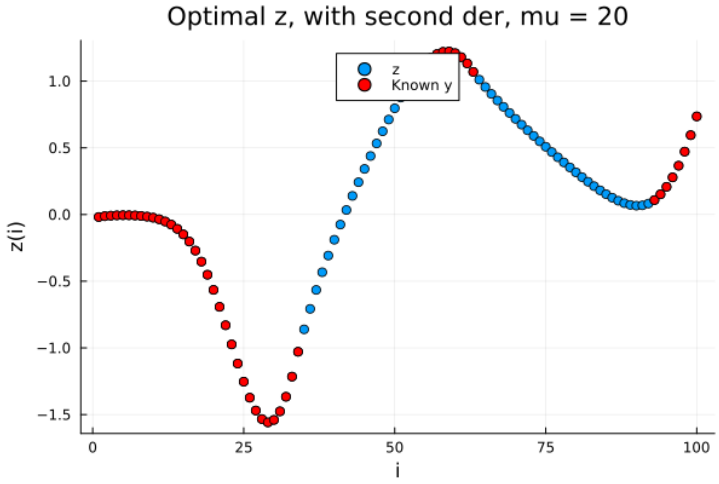


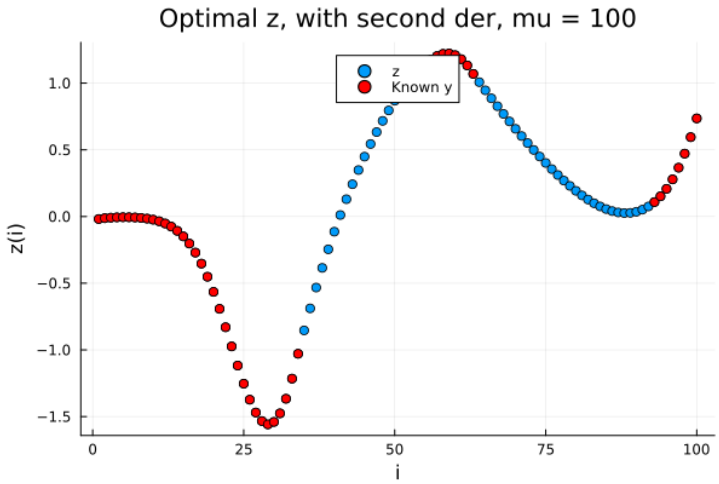
g.

This question was done in Julia, the code and plot can be seen below.

```julia
#Part E

mu_values_to_plot = [5, 20, 100]
for mu in mu_values_to_plot
    z_tradeoff_K_c = - inv(G_K_c' * G_K_c + mu * H_K_c' * H_K_c) * (G_K_c' * G_K + mu * H_K_c' * H_K)* y
    z_tradeoff_opt = zeros(n)
    z_tradeoff_opt[K] = y
    z_tradeoff_opt[K_c] = z_tradeoff_K_c
    scatter(1:n, z_tradeoff_opt, label="z", xlabel="i", ylabel="z(i)", title="Optimal z, with second der, mu = $mu", legend=:top)
    scatter!(K, y, label="Known y", color=:red)
    savefig("hw6_q3e_mu=$mu.png")
end
```


Optimal z, with second der, mu = 5
z
Known y
z(i)
i
1.0
0.5
0.0
−0.5
−1.0
−1.5
0
25
50
75
100


Optimal z, with second der, mu = 20
z
Known y
z(i)
i
1.0
0.5
0.0
−0.5
−1.0
−1.5
0
25
50
75
100


Optimal z, with second der, mu = 100
z
Known y
z(i)
i
1.0
0.5
0.0
−0.5
−1.0
−1.5
0
25
50
75
100

11.1830.)

a.

We start by finding a general matrix, call it B, with A's eigenvectors. We want to find the optimal eigenvalues for B which will give us $\hat{A}$. Let T be the matrix whose columns are the linearly independent eigenvectors of A and Λ be a diagonal matrix containing the corresponding eigenvalues. We can express T by its columns (which are just the eigenvectors of A) and T^{-1} by its rows:

$$T = \begin{bmatrix} v_1 & v_2 & ... & v_n \end{bmatrix}$$

$$T^{-1} = \begin{bmatrix} \tilde{t_1}^\intercal \\ \tilde{t_2}^\intercal \\ \vdots \\ \tilde{t_n}^\intercal \end{bmatrix}$$

Then we have

$$B = T\Lambda T^{-1} = \begin{bmatrix} v_1 & v_2 & ... & v_n \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 & ... & 0 \\ 0 & \lambda_2 & ... & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & ... & \lambda_n \end{bmatrix} \begin{bmatrix} \tilde{t_1}^\intercal \\ \tilde{t_2}^\intercal \\ \vdots \\ \tilde{t_n}^\intercal \end{bmatrix} = \sum_{i=1}^{n} \lambda_i v_i \tilde{t_i}^\intercal$$

Define $\tilde{B}$ to be a column vector consisting of all the entries of B i.e.

$$\tilde{B} = \begin{bmatrix} B_{11} \\ B_{21} \\ \vdots \\ B_{n1} \\ B_{12} \\ B_{22} \\ \vdots \\ B_{nn} \end{bmatrix}$$

Then we have

$$\tilde{B} = \begin{bmatrix} B_{11} \\ B_{21} \\ \vdots \\ B_{n1} \\ B_{12} \\ B_{22} \\ \vdots \\ B_{nn} \end{bmatrix} = C \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_n \end{bmatrix} = \begin{bmatrix} (v_1\tilde{t_1}^\intercal)_{11} & (v_2\tilde{t_2}^\intercal)_{11} & ... & (v_n\tilde{t_n}^\intercal)_{11} \\ (v_1\tilde{t_1}^\intercal)_{21} & (v_2\tilde{t_2}^\intercal)_{21} & ... & (v_n\tilde{t_n}^\intercal)_{21} \\ \vdots & \vdots & \ddots & \vdots \\ (v_1\tilde{t_1}^\intercal)_{n1} & (v_2\tilde{t_2}^\intercal)_{n1} & ... & (v_n\tilde{t_n}^\intercal)_{n1} \\ (v_1\tilde{t_1}^\intercal)_{12} & (v_2\tilde{t_2}^\intercal)_{12} & ... & (v_n\tilde{t_n}^\intercal)_{12} \\ (v_1\tilde{t_1}^\intercal)_{22} & (v_2\tilde{t_2}^\intercal)_{22} & ... & (v_n\tilde{t_n}^\intercal)_{22} \\ \vdots & \vdots & \ddots & \vdots \\ (v_1\tilde{t_1}^\intercal)_{nn} & (v_2\tilde{t_2}^\intercal)_{nn} & ... & (v_n\tilde{t_n}^\intercal)_{nn} \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_n \end{bmatrix}$$

The definition of $\tilde{A}^{\text{meas}}$ is analogous to $\tilde{B}$. Notice that minimizing J is equivalent to choosing $\lambda_1, \lambda_2, ..., \lambda_n$ to minimize the following

$$\min \|\tilde{B} - \tilde{A}^{\text{meas}}\|^2 = \min_{\lambda_1,\lambda_2,...,\lambda_n} \|C \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_n \end{bmatrix} - \tilde{A}^{\text{meas}}\|^2$$

We can solve for the eigenvalues using least squares assuming that C is skinny and full rank. If we insert these eigenvalues into a diagonal matrix Λ_{opt}, we can find $\hat{A}$:

$$\hat{A} = T\Lambda_{\text{opt}}T^{-1}$$

b.

This question was done in Julia, the code and results can be seen below.

```julia
using LinearAlgebra

#Question 4.)

Ameas = [2.0  1.2 -1.0;
         0.4  2.0 -0.5;
        -0.5  0.9  1.0]

v1 = [0.7, 0.0, 0.7]
v2 = [0.3, 0.6, 0.7]
v3 = [0.6, 0.6, 0.3]

T = hcat(v1, v2, v3)
invT = inv(T)
n = 3

C = zeros(n*n, n)
for i in 1:n
    e = zeros(n)
    e[i] = 1.0
    B_i = T * Diagonal(e) * invT
    C[:, i] = B_i[:]
end

Ameas_tilde = Ameas[:]
lambda_opt = C \ Ameas_tilde
Ahat = T * Diagonal(lambda_opt) * invT
eigvals_Ahat, eigvecs_Ahat = eigen(Ahat)
J = (1/(n*n)) * sum((Ameas - Ahat).^2)

println("Optimal Eigenvalues = ", lambda_opt)
println("Ahat = \n", Ahat)
println("Frobenius cost J = ", J)
println("Eigenvectors of Ahat = ", eigvecs_Ahat)
println("Eigenvalues of Ahat = ", eigvals_Ahat)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
TERMINAL

Optimal Eigenvalues = [0.7840290263886042, 1.80014104068647, 2.4158299329249253]
Ahat =
[1.7472389401492223 1.150195949656012 -0.9632099137606183; 0.5277333362043901 2.1519632648227303 -0.5277333362043901; -0.3167687686394427 0.9742848375878819 1.1007977950280468]
Frobenius cost J = 0.01746133343860544
Eigenvectors of Ahat = [0.7071067811865478 0.30942637387763633 -0.6666666666666662; 3.723575447618661e-16 0.6188527477552768 -0.6666666666666677; 0.7071067811865472 0.7219948723811553 -0.3333333333333322]
Eigenvalues of Ahat = [0.7840290263886041, 1.8001410406864697, 2.4158299329249266]
```

As can be seen in the output,

$$\hat{A} = \begin{bmatrix} 1.747 & 1.150 & -0.963 \\ 0.528 & 2.152 & -0.528 \\ -0.317 & 0.974 & 1.101 \end{bmatrix}$$

$$\begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix} = \begin{bmatrix} 0.784 \\ 1.800 \\ 2.416 \end{bmatrix}$$

$$J = 0.0175$$

We also verify that $\hat{A}$ has v_1, v_2, v_3 as eigenvectors by finding the eigenvectors and eigenvalues of $\hat{A}$. These eigenvectors and eigenvalues can be seen in the output of the code above. The outputted eigenvectors are approximately $v_1, v_2, -v_3$, which verifies that $\hat{A}$ indeed has v_1, v_2, v_3 as eigenvectors.

15.2200.)

a.

True.

$$P + Q \geq Q \Leftrightarrow (P + Q) - Q \geq 0 \Leftrightarrow P \geq 0$$

Hence, if $P \geq 0$ then $P + Q \geq Q$.

b.

True.

$$-P \leq -Q \Leftrightarrow -Q \geq -P \Leftrightarrow -Q - (-P) \geq 0 \Leftrightarrow P - Q \geq 0 \Leftrightarrow P \geq Q$$

Hence, if $P \geq Q$ then $-P \leq -Q$.

c.

True. Since P is symmetric, we can diagonalize P with orthogonal V,

$$P = V\Lambda V^\intercal$$

This means that we can diagonalize P^{-1} with orthogonal $V^\intercal$

$$P^{-1} = V\Lambda^{-1}V^\intercal$$

Because

$$PP^{-1} = P^{-1}P = V\Lambda V^\intercal V\Lambda^{-1}V^\intercal = V\Lambda^{-1}V^\intercal V\Lambda V^\intercal = I$$

This is all to say that if Λ is the diagonal matrix of eigenvalues of P then Λ^{-1} is the diagonal matrix of eigenvalues of P^{-1}, so the eigenvalues of P^{-1} are the positive reciprocals of the eigenvalues of P. We know that a matrix $A > 0$ if and only if the eigenvalues of A are all positive. Since $P > 0$, we know that the eigenvalues of P are all positive, and since the eigenvalues of P^{-1} are the positive reciprocals of the eigenvalues of P, we have that the eigenvalues of P^{-1} are all positive. Hence, $P^{-1} > 0$.

d.

True. We want to show that for any $x \in \mathbb{R}^n$,

$$x^\intercal Q^{-1}x \geq x^\intercal P^{-1}x$$

First some asides. We'll show that $Z \geq I$ implies $Z^{-1} \leq I$. Assuming that $Z \geq I$, we have for every eigenvalue λ and eigenvector v of Z,

$$v^\intercal Zv \geq v^\intercal Iv \Rightarrow \lambda\|v\|^2 \geq \|v\|^2 \Rightarrow \lambda \geq 1$$

So if $Z \geq I$ then the eigenvalues of Z must all be greater than or equal to 1. Since the eigenvalues of Z^{-1} are the reciprocals of the eigenvalues of Z (shown in part c), that means the eigenvalues of Z^{-1} must all be less than or equal to 1. Let $\tilde{\lambda}$ be the maximum of the eigenvalues of Z^{-1}. We then have for any vector x,

$$x^\intercal Z^{-1}x \leq \tilde{\lambda}\|x\|^2 \leq \|x\|^2 = x^\intercal Ix \Rightarrow Z^{-1} \leq I$$

Thus, $Z \geq I$ implies $Z^{-1} \leq I$. Note that $Q = V\Lambda V^\intercal$ for some diagonal Λ and orthogonal V. Now define

$$Q^{1/2} = V\Lambda^{1/2}V^\intercal$$

Where $\Lambda^{1/2}$ is a diagonal matrix whose entries are the square roots of the eigenvalues in Λ. We can do this because $Q > 0$ implies that all of Q's eigenvalues are greater than 0. We can define $Q^{-1/2}$ in a similar way. We get the following nice properties.

$$Q^{1/2} \text{ is invertible as all of its eigenvalues are greater than zero}$$

$$(Q^{1/2})^{-1} = Q^{-1/2}$$

$$(Q^{1/2})^2 = Q, (Q^{-1/2})^2 = Q^{-1}$$

Now define

$$Z = Q^{-1/2}PQ^{-1/2}$$

This would imply $Z^{-1} = Q^{1/2}P^{-1}Q^{1/2}$. Then

$$Z - I = Q^{-1/2}(P - Q)Q^{-1/2}$$

Consider arbitrary $x \in \mathbb{R}^n$. As $Q^{1/2}$ is invertible, we know there exists unique y such that $x = Q^{1/2}y$. Then we have

$$x^\intercal(Z - I)x = y^\intercal Q^{1/2}Q^{-1/2}(P - Q)Q^{-1/2}Q^{1/2}y = y^\intercal(P - Q)y \geq 0$$

Therefore $Z - I \geq 0$, so $Z \geq I$ and $Z^{-1} \leq I$. This means for any $x \in \mathbb{R}^n$. we have

$$x^\intercal Z^{-1}x = x^\intercal Q^{1/2}P^{-1}Q^{1/2}x \leq ||x||^2 = x^\intercal Ix$$

Using all this, we'll show that for any $x \in \mathbb{R}^n$,

$$x^\intercal Q^{-1}x \geq x^\intercal P^{-1}x$$

Consider arbitrary $x \in \mathbb{R}^n$. As $Q^{1/2}$ is invertible, we know there exists unique y such that $x = Q^{1/2}y$. Then we have

$$x^\intercal Q^{-1}x \geq x^\intercal P^{-1}x \Leftrightarrow y^\intercal Q^{1/2}Q^{-1}Q^{1/2}y \geq y^\intercal Q^{1/2}P^{-1}Q^{1/2}y$$

$$\Leftrightarrow \|y\|^2 \geq y^\intercal Q^{1/2}P^{-1}Q^{1/2}y$$

We've proven $\|y\|^2 \geq y^\intercal Q^{1/2}P^{-1}Q^{1/2}y$ holds, thus $x^\intercal Q^{-1}x \geq x^\intercal P^{-1}x$ for all x which means $Q^{-1} \geq P^{-1}$.

e.

False. Let

$$P = \begin{bmatrix} 1 \end{bmatrix}$$

$$Q = \begin{bmatrix} -2 \end{bmatrix}$$

Then

$$P - Q = \begin{bmatrix} 3 \end{bmatrix} \Rightarrow P - Q \geq 0 \Rightarrow' P \geq Q$$

However

$$P^2 - Q^2 = \begin{bmatrix} -3 \end{bmatrix} \Rightarrow P^2 - Q^2 \ngeq 0 \Rightarrow' P^2 \ngeq Q^2$$

We've thus found symmetric P and Q where $P \geq Q$ and $P^2 \ngeq Q^2$, completing our counterexample.